15/04/26

Métodos de Pesquisa

→ Interna → mais rápido, mas há casos que memória externa seja necessário

→ Pode ser usar métodos lineares (Pesquisa binária (sequencial)) ou espacial (árvore)

→ Quem considerar: volume de dados, inserção x retirados frequente, estabilidade

$O(n)$ → - **Sequencial Rápida**: percorre toda a tabela, utilizando sentinela;

$O(\log_2 n)$ - **Pesquisa Binária**: subdivisão do espaço de busca, reduzindo-o pela metade;

$O(\log n)$ - **Árvore Binária de Busca (ABB):** subdivisão do espaço de busca, reduzindo-o pela metade.
  - **Não Balanceada**
  - **Balanceada**

- **caso médio:** $f(n) = \frac{(n+1)}{2}$
  - Considerar que toda pesquisa recupera um registro.
  - Se p_i é a probabilidade de se encontrar o i-ésimo registro, logo:
    - $f(n) = 1 \,.\, p_1 + 2 \,.\, p_2 + 3 \,.\, p_3 + \ldots + n \,.\, p_n$.
  - Para calcular $f(n)$
    - Considerar uma distribuição de probabilidades p_i (Igualmente Provável)
    - Neste caso:

$$p_i = \frac{1}{n}, 1 \le i \le n.$$

$$f(n) = 1.\frac{1}{n} + 2.\frac{1}{n} + 3.\frac{1}{n} + \ldots n.\frac{1}{n}$$

$$f(n) = \frac{1}{n}.(1+2+3+\ldots+n) \quad \textbf{(PA)}$$

$$f(n) = \frac{1}{n}.(\frac{n.(n+1)}{2})$$

$$f(n) = \frac{(n+1)}{2}$$

linear

⇒ Parte 3 e 4 que vão precisar de ordenação

⇒ Mais rápido

Pesquisa binária

↳ precisa estar ordenado

↳ divide e compara varias vezes

- Seja g uma função de complexidade
  - $g(n)$ denota **tempo: Altamente dependente do espaço de busca**
- **Em cada divisão, o espaço de busca é reduzido:**
  - $n/2^1, n/2^2, n/2^3, \ldots, n/2^k$ (k – ésima divisão)
  - Em algum momento, o espaço de busca é 1:
    - Logo, tem-se $n/2^k = 1 \Rightarrow n = 2^k \Rightarrow \log_2 n = k$.
    - $g(n) = O(\log n)$

↳ muito mais ñ é ordenação do arquivo

- **Nas condições apresentadas, tem-se (tempo):**

	Ordenação	Pesquisa	Total (Tempo)
Melhor caso:	$O(n)$	$O(1)$	$O(n)$
Pior caso:	$O(n^2)$	$O(\log_2 n)$	$O(n^2)$

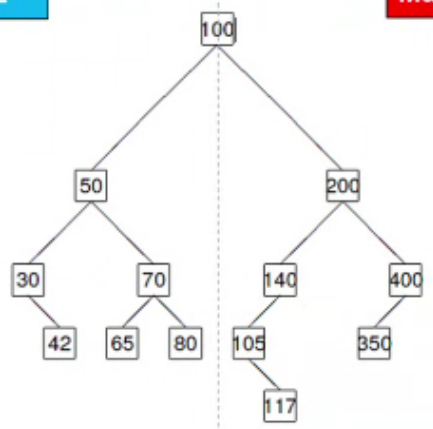
- **Logo,** Pesquisa Binária não é indicada para aplicações dinâmicas (com muitas mudanças);
- **Solução: Árvore Binária de Busca (ABB);**

olhar → para estrutura dinâmica

Árvore Binária de Busca

- **100 200 140 50 105 70 80 30 400 350 117 80 42 65**

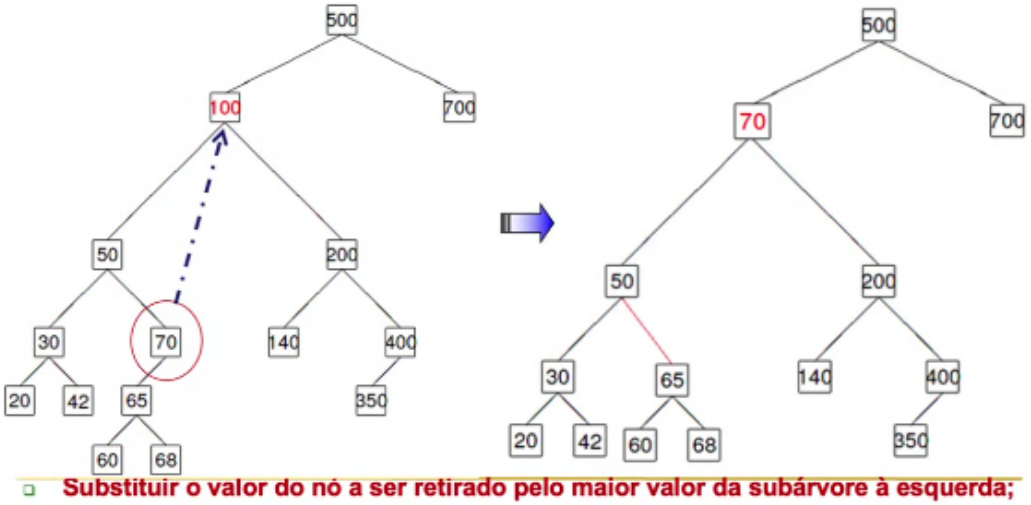
Menores que raiz ⟷ Maiores que raiz



mais à menor

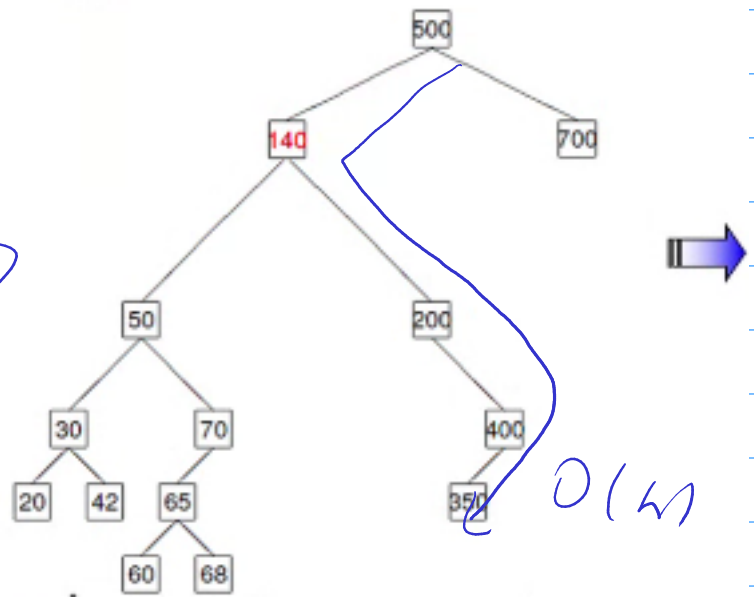
← mais à maior

Remoção de nó com 2 filhos



- Substituir o valor do nó a ser retirado pelo maior valor da subárvore à esquerda; ou,

ou ainda



O(n)

arvore degenerada

pior caso

O(n)

→ Inserção e remoção tendem a degenerar a árvore

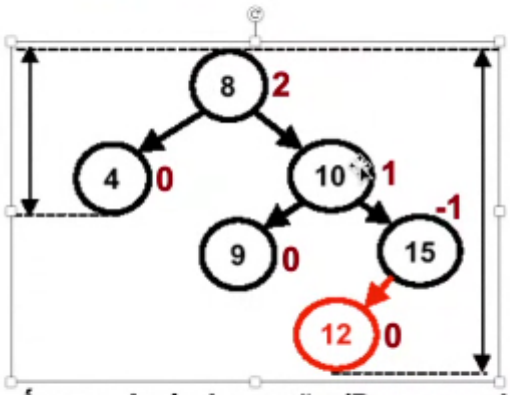
→ A maior função é melhor
+ Para melhor é preciso balanceamento

→ AVL → diferença de alturas
max ±1 (fator de balanceamento)
↳ cada nó armazena o valor do fator de balanc. e o nó pai (pointer)

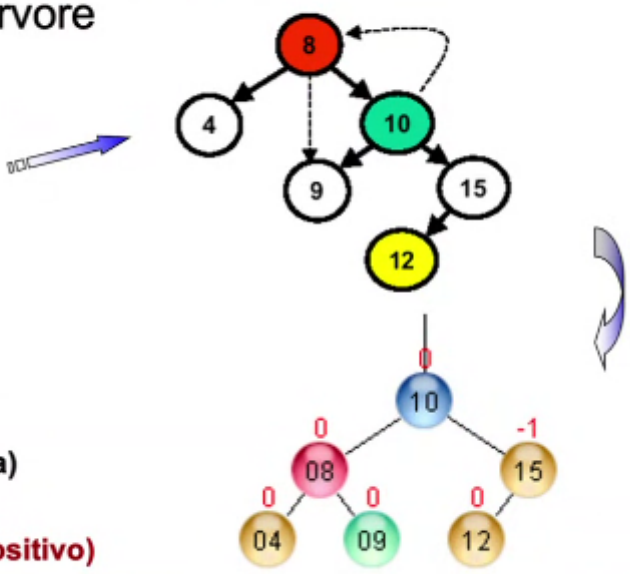
↳ se FB é negativo rotação a direita
↳ se não rotação a esquerda

Métodos de Pesquisa

- AVL: Balanceamento (Exemplo 1)
  - Inserir o nó **12** na árvore



Árvore Após Inserção (Degenerada)

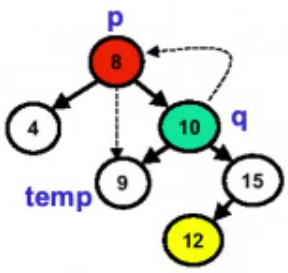


Árvore Balanceada

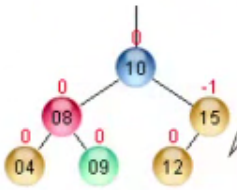
FB(8) = 3-1 = 2
FB(10) = 2-1 = 1
} Mesmo Sinal (positivo)

Rotação Simples à Esquerda sobre o nó com FB maior: 2

Esquema Rotação à Esquerda



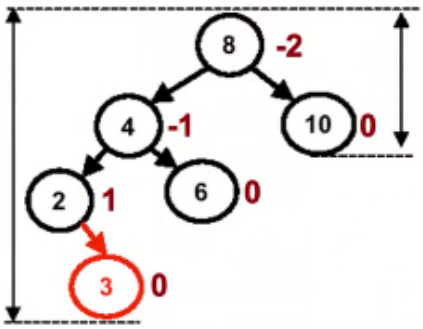

```
void rotacao_esquerda(AVL *p)
{
    AVL *q, *temp;
    q = p->dir;
    temp = q->esq;
    q->esq = p;
    p->dir = temp;
}
```



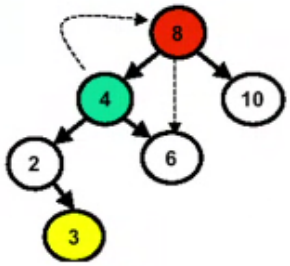
Árvore Balanceada

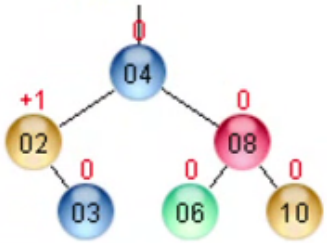
AVL: Balanceamento (Exemplo 2)

- Inserir o nó **3** na árvore



Árvore Após Inserção (Degenerada)





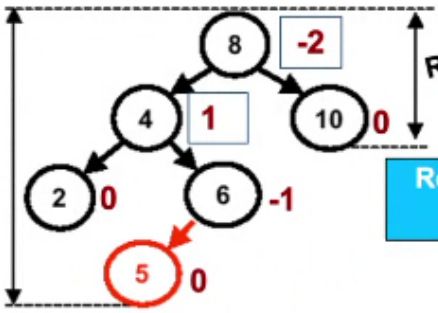
Árvore Balanceada

B(8) = 1-3 = -2
B(4) = 1-2 = -1
} Mesmo Sinal (negativo)

Rotação Simples à Direita sobre o nó com FB menor: -2

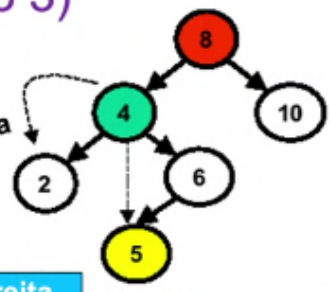
Métodos de Pesquisa

- AVL: Balanceamento (Exemplo 3)
  - Inserir o nó **5** na árvore

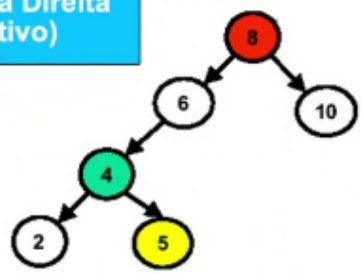


Árvore Após Inserção (Degenerada)

Rotação à Esquerda



Rotação Dupla à Direita
(Sinal Negativo)



Árvore Após Rotação Simples

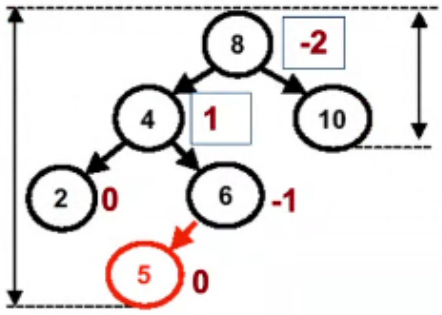
FB(8) = 1-3= -2
FB(4) = 2-1 = 1
} Sinais Diferentes

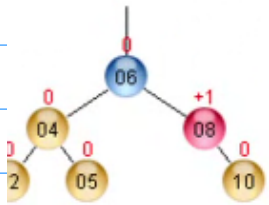
Rotação Simples à Esquerda sobre o nó com FB maior: 1

negativo = mais a esquerda
positivo = mais a direita

Métodos de Pesquisa

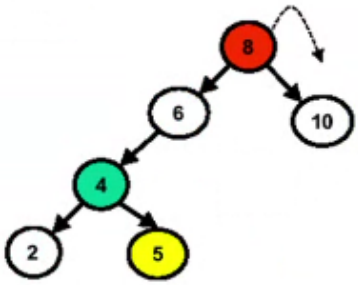
- AVL: Balanceamento (Exemplo 3)





Árvore Balanceada

FB(8) = 1-3 = -2



Árvore Após Rotação Simples

Rotação Simples à Direita sobre o nó com FB menor: -2

Considerações sobre as Operações: Árvore AVL

- **Inserção, Remoção e Pesquisa:**
  - Custo Altamente dependente do Percurso Realizado (altura h):
  - ABB Balanceada, considerando a abordagem AVL;
    - Recorrência: $T(n) = T(n / 2)$, $T(1) = 1$.

- **Solução: Método da Expansão Telescópica**
  - **Expandir a recorrência até detectar o comportamento**

1. $T(n) = T(n / 2^1)$
2. $T(n) = (T(n / 2^2))$
3. $T(n) = ((T(n / 2^3)))$
4. $T(n) = (\ldots T((T(n / 2^h))) \ldots)$
5. $T(n) = n/2^h = 1$
6. $T(n) = n = 2^h \Rightarrow h = \log_2 n. \Longrightarrow T(n) = O(\log n)$

- **Nas condições apresentadas, tem-se (tempo):**

	Rotação	Pesquisa	Total (Tempo)
Melhor caso:	$O(1)$	$O(1)$	$O(1)$
Pior caso:	$O(1)$	$O(\log n)$	$O(\log n)$

- **Logo,** ABB é indicada para aplicações **DINÂMICAS**.